C++ Level 1 – Interview Revision Notes (Q1–Q40)

Compile with: `g++ -std=c++17 -Wall -Wextra -Wpedantic file.cpp -o program`

Legend: ✅ = your code was correct (kept as is) · 🐞 = bug / spec mismatch found and fixed

Quick Index

#	Topic	Status
1	Hello World	🐞 text mismatch
2	Sum of two numbers	🐞 overflow risk
3	Even or odd	✅
4	Largest of two	✅
5	Largest of three	✅
6	Simple calculator	🐞 input order + decimals
7	Leap year	✅
8	Multiplication table	✅
9	Sum 1 to N	🐞 no loop, overflow
10	Reverse integer	✅
11	Count digits	✅
12	Prime check	✅
13	Factorial	✅
14	Fibonacci	🐞 trailing space
15	Sum of digits	✅
16	Armstrong	🐞 overflow on huge input
17	Palindrome	✅
18	GCD	✅
19	LCM	🐞 missing `<climits>`

Q1. Print Hello, World!

Bug 🐞: The question asks for `Hello, World!` (comma + capital W). Your code printed `Hello world!`. Judges and interviewers compare output exactly.

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

Explanation

- `#include <iostream>` gives access to `std::cout`.
- `<<` is the stream insertion operator; it sends text to the console.
- `std::endl` prints a newline and flushes the buffer. `'\n'` is faster; use it in loops with lots of output.

Output: `Hello, World!`

Interview tip: "What's the difference between `endl` and `'\n'`?" → `endl` = newline + flush.

Q2. Sum of Two Integers

Bug 🐞: `int` overflows when the sum exceeds ~2.1 billion (e.g. 2,000,000,000 + 2,000,000,000 is undefined behavior for signed int). Use `long long`.

```cpp
#include <iostream>

int main() {
    long long a, b;
    std::cin >> a >> b;
    std::cout << a + b << '\n';
    return 0;
}
```

Example

```
Input : 12 8
Output: 20
```

Explanation: `cin >> a >> b` reads two whitespace-separated values. Signed integer overflow is *undefined behavior* in C++, not just a wrong answer.

Q3. Even or Odd ✅

cpp

```cpp
#include <iostream>

int main() {
    int num;
    std::cout << "Enter the number: ";
    std::cin >> num;

    if (num % 2 == 0)
        std::cout << "Even" << std::endl;
    else
        std::cout << "Odd" << std::endl;

    return 0;
}
```

Explanation: `num % 2` is the remainder after dividing by 2. 0 → even.

Examples

Input	Output
17	Odd
10	Even
-3	Odd (works because `-3 % 2 == -1`, which is `!= 0`)

Interview tip: Never test oddness with `num % 2 == 1`. It fails for negatives (`-3 % 2 == -1`). Test `!= 0`, or use `num & 1`.

Q4. Largest of Two Numbers ✅

cpp

```cpp
#include <iostream>

int main() {
    int a, b;
    std::cout << "Enter the number 1: ";
    std::cin >> a;
    std::cout << "Enter the number 2: ";
    std::cin >> b;

    if (a > b)
        std::cout << a << " is greater than " << b << std::endl;
    else if (b > a)
        std::cout << b << " is greater than " << a << std::endl;
    else
        std::cout << "Equal" << std::endl;

    return 0;
}
```

Explanation: Three mutually exclusive cases: `a > b`, `b > a`, or neither (equal).

Input	Output
9 9	Equal
3 8	8 is greater than 3

Q5. Largest of Three Numbers ✅

cpp

```cpp
#include <iostream>

int main() {
    int a, b, c;
    std::cout << "Enter three numbers: ";
    std::cin >> a >> b >> c;

    if (a >= b && a >= c)
        std::cout << a << " is the largest\n";
    else if (b >= a && b >= c)
        std::cout << b << " is the largest\n";
    else
        std::cout << c << " is the largest\n";

    return 0;
}
```

Explanation: Using `>=` (not `>`) correctly handles ties. With `5 5 3`, `a >= b && a >= c` is true so `5` is printed. If you used `>`, no branch might match.

Input	Output
14 27 19	27
5 5 3	5
7 7 7	7

Alternative: `int m = a; if (b > m) m = b; if (c > m) m = c;`

Q6. Simple Calculator

Bugs 🐞

1. The question's example is `Input: 10 2 /` (numbers first, then operator). Your code read the operator first, so the example input would fail.
2. Used `int`, so `7.5 + 2` would break. Use `double`.
3. Division by zero for doubles doesn't crash (gives `inf`), so you must check explicitly. You already did, which is good.

```cpp
```

```cpp
#include <iostream>

int main() {
    double a, b;
    char op;
    std::cin >> a >> b >> op;

    switch (op) {
    case '+':
        std::cout << a + b << '\n';
        break;
    case '-':
        std::cout << a - b << '\n';
        break;
    case '*':
        std::cout << a * b << '\n';
        break;
    case '/':
        if (b == 0)
            std::cout << "Error: division by zero\n";
        else
            std::cout << a / b << '\n';
        break;
    default:
        std::cout << "Invalid operator\n";
    }
    return 0;
}
```

Examples

```
Input : 10 2 /      Output: 5
Input : 7 0 /       Output: Error: division by zero
Input : 3 4 %       Output: Invalid operator
Input : 2.5 2 *     Output: 5
```

Explanation: `switch` jumps to the matching `case`. `break` stops it falling through to the next case. `default` handles anything else.

Interview tip: Integer `a / b` truncates (`7/2 == 3`). Cast to `double` (or use double variables) for real division.

Q7. Leap Year ✅

cpp

```cpp
#include <iostream>

int main() {
    int year;
    std::cout << "Enter a year: ";
    std::cin >> year;

    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
        std::cout << "Leap year\n";
    else
        std::cout << "Not leap year\n";

    return 0;
}
```

Rule (Gregorian): divisible by 4 **and** not by 100, **or** divisible by 400.

Year	Result	Why
2000	Leap	divisible by 400
1900	Not leap	divisible by 100 but not 400
2024	Leap	divisible by 4, not 100
2023	Not leap	not divisible by 4

Q8. Multiplication Table ✅

cpp

```cpp
#include <iostream>

int main() {
    int a;
    std::cout << "Enter number: ";
    std::cin >> a;

    for (int i = 1; i <= 10; i++)
        std::cout << a << " * " << i << " = " << a * i << '\n';

    return 0;
}
```

(Small polish: I printed `a * i` so the table reads naturally as `5 * 1 = 5`; your `i * a` is mathematically identical.)

Example (input 5):

```
5 * 1 = 5
5 * 2 = 10
...
5 * 10 = 50
```

Explanation: `for(init; condition; update)`: starts at 1, runs while `i <= 10`, increments each loop.

Q9. Sum of First N Natural Numbers

Bugs 🐞

1. The question says *"using a loop"*. Your code used the formula `n(n+1)/2`. The formula is great in an interview (as the optimization), but show the loop first.
2. `n * (n + 1)` can overflow before dividing by 2 if `n` is very large.
3. Negative `n` isn't handled.

cpp

```cpp
#include <iostream>

int main() {
    long long n;
    std::cin >> n;

    if (n < 0) {
        std::cout << "Enter a non-negative number\n";
        return 0;
    }

    // Loop approach: O(n)
    long long sum = 0;
    for (long long i = 1; i <= n; i++)
        sum += i;
    std::cout << "Sum (loop)    = " << sum << '\n';

    // Formula approach: O(1)
    // Divide the even factor first to reduce overflow risk.
    long long formula = (n % 2 == 0) ? (n / 2) * (n + 1)
                                     : n * ((n + 1) / 2);
    std::cout << "Sum (formula) = " << formula << '\n';

    return 0;
}
```

Example

```
Input : 5
Output: Sum (loop) = 15, Sum (formula) = 15
```

Explanation: Gauss's trick: pair first and last (1+5, 2+4, ...) so you get `n/2` pairs each summing to `n+1`. Dividing the even factor *before* multiplying avoids the intermediate overflow.

Complexity: loop O(n), formula O(1). Both O(1) memory.

Q10. Reverse an Integer ✅

cpp

```cpp
#include <iostream>
#include <climits>
using namespace std;

bool reverseInteger(int num, int &result) {
    long long reversed = 0;
    int sign = (num < 0) ? -1 : 1;
    long long n = num;          // widen first so -INT_MIN is safe
    if (n < 0) n = -n;

    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }
    reversed *= sign;

    if (reversed < INT_MIN || reversed > INT_MAX)
        return false;           // overflow

    result = static_cast<int>(reversed);
    return true;
}

int main() {
    int num, result;
    cin >> num;

    if (reverseInteger(num, result))
        cout << result << '\n';
    else
        cout << "Overflow\n";
}
```

Dry run (1234):

n	n % 10	reversed
1234	4	4
123	3	43
12	2	432
1	1	4321

Examples

Input	Output
1234	4321
1200	21 (trailing zeros vanish)
1534236469	Overflow (answer 9646324351 > INT_MAX)
-123	-321

Why it's good: Storing the answer in `long long` lets you *detect* the overflow instead of silently wrapping. A 32-bit int has at most 10 digits, and its reversal always fits in `long long`.

Interview tip: Alternative without `long long`: check `if (rev > INT_MAX/10 || (rev == INT_MAX/10 && digit > 7)) overflow` before each step.

Q11. Count the Digits ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long num;
    cin >> num;

    int count = 0;
    do {
        count++;
        num /= 10;
    } while (num != 0);

    cout << count << '\n';
}
```

Why `do-while`: it runs the body at least once, so `0` correctly gives `1`. A plain `while (num > 0)` would give `0` digits for input `0`.

Why negatives work (bonus): C++11+ integer division truncates toward zero, so `-123 / 10 == -12`, `-12 / 10 == -1`, `-1 / 10 == 0`, and the loop ends with `!= 0`. No `abs()` needed.

Input	Output
12345	5
7	1
0	1
-987	3

Q12. Check for a Prime Number ✅

cpp

```cpp
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;                  // 2 and 3
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i <= n / i; i += 6) {     // i*i <= n without overflow
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    }
    return true;
}

int main() {
    int n;
    cin >> n;
    cout << (isPrime(n) ? "Prime" : "Not prime") << '\n';
}
```

Explanation

- Primes < 2 don't exist (0, 1, negatives).
- Every prime > 3 has the form `6k ± 1`, so test only `i` and `i+2` while stepping by 6. That's about 3x fewer checks than testing every number.
- We only need divisors up to √n, because if `n = a*b` then one of `a, b` is ⩽ √n.
- `i <= n / i` is the overflow-safe form of `i * i <= n`.

Input	Output
29	Prime
1	Not prime
49	Not prime (7 × 7)
2	Prime

Complexity: O(√n).

Q13. Factorial ✅

cpp

```cpp
#include <iostream>
#include <climits>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n < 0) {
        cout << "Factorial is undefined for negative numbers\n";
        return 0;
    }

    long long fact = 1;                      // handles 0! = 1 automatically
    for (int i = 2; i <= n; i++) {
        if (fact > LLONG_MAX / i) {          // would fact * i overflow?
            cout << "Overflow\n";
            return 0;
        }
        fact *= i;
    }
    cout << fact << '\n';
}
```

Explanation

- `fact` starts at 1, so for `n = 0` or `1` the loop never runs and the answer is 1.
- **Overflow guard:** `fact * i > LLONG_MAX` is the same as `fact > LLONG_MAX / i`. Check *before* multiplying, since overflow after the fact is already undefined behavior.

- Largest that fits in `long long`: `20! = 2,432,902,008,176,640,000`. `21!` prints `Overflow`.

Input	Output
5	120
0	1
20	2432902008176640000
21	Overflow

Q14. Fibonacci Sequence

Bug 🐞 (cosmetic): the original printed a trailing space after the last number (`0 1 1 2 3 5 `). Fixed by printing a separator only *before* every number except the first.

cpp

```cpp
#include <iostream>
#include <climits>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    long long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << ' ';
        cout << a;

        if (i < n - 1) {
            if (a > LLONG_MAX - b) {          // a + b would overflow
                cout << "\nOverflow\n";
                return 0;
            }
            long long next = a + b;
            a = b;
            b = next;
        }
    }
    cout << '\n';
}
```

Dry run (n = 6):

i	print a	next = a+b	(a, b) after
0	0	1	(1, 1)
1	1	2	(1, 2)
2	1	3	(2, 3)
3	2	5	(3, 5)
4	3	8	(5, 8)
5	5	(skipped, last)	

Output: `0 1 1 2 3 5`

Explanation: Keep only the last two values, so O(1) memory. The `i < n - 1` guard avoids computing one unneeded term (which could overflow for no reason). The `long long` limit is about 92 terms.

Q15. Sum of Digits ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) n = -n;

    int sum = 0;
    do {
        sum += static_cast<int>(n % 10);
        n /= 10;
    } while (n > 0);

    cout << sum << '\n';
}
```

Dry run (583): 583 % 10 = 3 → sum 3; 58 % 10 = 8 → 11; 5 % 10 = 5 → **16.**

Input	Output
583	16
0	0
-45	9

Core idea: `% 10` extracts the last digit, `/ 10` removes it. This pair is the foundation of Q10, 11, 15, 16, 17, 21, 23, 24 and 32.

Q16. Armstrong Number

Bug 🐞 (edge case): For very large inputs (up to 19 digits), `sum` can overflow `long long` because each term can be ~`9^19`. Fix: stop early once `sum > n`, since the sum only grows.

cpp

```cpp
#include <iostream>
using namespace std;

long long integerPower(int base, int exp) {
    long long result = 1;
    while (exp-- > 0)
        result *= base;
    return result;
}

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Not an Armstrong number\n";
        return 0;
    }

    // Step 1: count digits
    long long temp = n;
    int digits = 0;
    do {
        digits++;
        temp /= 10;
    } while (temp > 0);

    // Step 2: sum of digit^digits
    temp = n;
    long long sum = 0;
    do {
        int digit = static_cast<int>(temp % 10);
        sum += integerPower(digit, digits);
        if (sum > n) break;                // 🐞 fix: stop before overflow
        temp /= 10;
    } while (temp > 0);

    cout << (sum == n ? "Armstrong number" : "Not an Armstrong number") << '\n'
}
```

Dry run (153): digits = 3 → $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = \mathbf{153}$ ✓

Input	Output
153	Armstrong number
9474	Armstrong number ($9^4+4^4+7^4+4^4$)
123	Not an Armstrong number
0	Armstrong number ($0^1 = 0$)

Interview tip: Don't use `std::pow` for this. It works in `double` and may return `124.99999` → cast to `124`. A hand-written integer loop is exact.

Q17. Palindrome Number ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n < 0) {
        cout << "Not a palindrome\n";
        return 0;
    }

    int original = n;
    long long reversed = 0;              // long long: reversal can exceed int
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }

    cout << (reversed == original ? "Palindrome" : "Not a palindrome") << '\n';
}
```

Example (1221): reversed = 1221 == original → **Palindrome.**

Input	Output
1221	Palindrome
123	Not a palindrome
0	Palindrome
10	Not a palindrome (reverses to 1)

Why `long long reversed`: e.g. `1999999999` reverses to `9999999991`, which overflows `int`. The wider type avoids a false result.

Q18. GCD (Euclidean Algorithm) ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long a, b;
    cin >> a >> b;

    if (a < 0 || b < 0) {
        cout << "Enter non-negative numbers\n";
        return 0;
    }

    while (b != 0) {
        long long remainder = a % b;
        a = b;
        b = remainder;
    }
    cout << a << '\n';
}
```

Idea: `gcd(a, b) = gcd(b, a % b)`, and `gcd(a, 0) = a`.

Dry run (48, 18):

a	b	a % b
48	18	12
18	12	6
12	6	0
6	0	stop → **6**

Input	Output
48 18	6
7 0	7
0 0	0 (by convention)

Complexity: O(log(min(a, b))). In real code, use `std::gcd` from `<numeric>` (C++17).

Q19. LCM

Bug 🐞: The code uses `LLONG_MAX` but never includes `<climits>`. It compiles on some compilers only because `<iostream>` happens to pull it in; on others it's a compile error. Always include what you use.

cpp

```cpp
#include <iostream>
#include <climits>      // 🐞 fix: needed for LLONG_MAX
using namespace std;

long long gcd(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    long long a, b;
    cin >> a >> b;

    if (a < 0 || b < 0) {
        cout << "Enter non-negative numbers\n";
        return 0;
    }

    if (a == 0 || b == 0) {            // LCM with zero is 0 (and avoids /0)
        cout << 0 << '\n';
        return 0;
    }

    long long g = gcd(a, b);
    long long reduced = a / g;         // divide FIRST to keep numbers small

    if (reduced > LLONG_MAX / b) {     // would reduced * b overflow?
        cout << "Overflow\n";
        return 0;
    }

    cout << reduced * b << '\n';
}
```

Formula: `lcm(a, b) = (a / gcd(a, b)) * b`. Dividing before multiplying is the overflow-reduction trick; `a * b / g` could overflow even when the final answer fits.

Dry run (4, 6): gcd = 2 → reduced = 4 / 2 = 2 → 2 × 6 = **12**.

Input	Output
4 6	12
5 0	0
7 13	91
9000000000000000000 9000000000000000001	Overflow

Note: A name clash is possible, as `gcd` can conflict with `std::gcd` when combined with `using namespace std;` in C++17 if `<numeric>` is included. If that happens, rename it to `myGcd`.

Summary of All Fixes

Q	Problem	Fix
1	Printed `Hello world!` instead of `Hello, World!`	Corrected the string
2	`int` overflow	`long long`
6	Read operator before numbers; integers only	Read `a b op`; use `double`
9	Formula used instead of loop; overflow; negatives	Added loop, safe formula, input check
16	`sum` could overflow on 19-digit input	Early `break` when `sum > n`
19	`LLONG_MAX` used without `<climits>`	Added `#include <climits>`

Patterns to Remember for Interviews

1. `% 10` **and** `/ 10` extract and drop the last digit (Q10, 11, 15, 16, 17).
2. `do-while` when the input `0` must still run once (Q11, 15, 16).
3. **Check overflow *before* the operation:** `a > MAX / b`, `a > MAX - b` (Q13, 14, 19).
4. **Loop up to √n** using `i <= n / i` (Q12).
5. **Euclid:** `gcd(a, b) = gcd(b, a % b)`; `lcm = a / gcd * b` (Q18, 19).
6. **Widen the type** (`int` → `long long`) for intermediate results (Q2, 10, 17).

PART 2 – Q20 to Q37

Legend: ✅ = your code was correct (kept as is) · 🐞 = bug / spec mismatch found and fixed

Quick Index (Part 2)

#	Topic	Status
20	Perfect number	✅
21	Strong number	✅
22	Power without `pow()`	🐞 wrong output format, slow, overflow
23	Count even/odd digits	✅
24	First and last digit	✅
25	Swap two numbers	🐞 missing "without temp" part
26	Optimized prime check	✅ (explanation added)
27	Primes in a range	✅
28	Prime factorization	🐞 trailing space
29	Decimal to binary	✅
30	Binary to decimal	✅
31	Power of two	✅
32	Digital root	✅
33	Right-angled star triangle	🐞 extra spaces vs expected output
34	Inverted star triangle	🐞 extra spaces
35	Number triangle	🐞 extra spaces
36	Repeated-number triangle	🐞 extra spaces
37	Pyramid	✅

Q20. Perfect Number ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n <= 1) {
        cout << "Not a perfect number\n";
        return 0;
    }

    long long sum = 1;                          // 1 divides every n > 1

    for (long long i = 2; i <= n / i; i++) { // only up to sqrt(n)
        if (n % i == 0) {
            sum += i;
            long long other = n / i;          // the paired divisor
            if (other != i)                   // don't add a square root twice
                sum += other;
        }
    }

    cout << (sum == n ? "Perfect number" : "Not a perfect number") << '\n';
}
```

Idea: Divisors come in pairs `(i, n/i)`. Looping only to √n finds both halves of every pair. This makes it O(√n) instead of O(n).

Dry run (28): sum starts at 1 → i=2: +2 +14 = 17 → i=4: +4 +7 = 28 → i=5: no → loop ends (5 > 28/5 = 5? stops when i > 5) → **28 == 28 → Perfect.**

Input	Output
6	Perfect number (1+2+3)
28	Perfect number
12	Not perfect (1+2+3+4+6 = 16)
1	Not perfect (no proper divisors)

Why `other != i`? For `n = 16`, at `i = 4` the pair is (4, 4). Adding it twice would be wrong.

Fact for interviews: The first perfect numbers are 6, 28, 496, 8128, 33550336.

Q21. Strong Number ✅

cpp

```cpp
#include <iostream>
using namespace std;

int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++)
        result *= i;
    return result;                // 0! and 1! return 1 (loop never runs)
}

int main() {
    long long n;
    cin >> n;

    if (n <= 0) {
        cout << "Not a strong number\n";
        return 0;
    }

    long long temp = n;
    long long sum = 0;

    while (temp > 0) {
        int digit = static_cast<int>(temp % 10);
        sum += factorial(digit);   // digit is 0..9, so 9! = 362880 fits in int
        temp /= 10;
    }

    cout << (sum == n ? "Strong number" : "Not a strong number") << '\n';
}
```

Dry run (145): $1! + 4! + 5! = 1 + 24 + 120 = \mathbf{145}$ ✓

Input	Output
145	Strong number
40585	Strong number (24+1+120+40320+120)
1, 2	Strong number (1! = 1, 2! = 2)
123	Not strong (1+2+6 = 9)

Why no overflow worry: digits are only 0–9, so the largest factorial needed is `9! = 362,880`.

Optimization to mention: Precompute `fact[10]` once instead of recomputing the factorial per digit.

```cpp
const int fact[10] = {1,1,2,6,24,120,720,5040,40320,362880};
```

Q22. Power Without `std::pow()`

Bugs 🐞

1. **Wrong-looking output:** with `double`, `cout` prints 6 significant digits, so `2^20` prints `1.04858e+06` instead of `1048576`. Interviewers will notice.
2. **Overflow isn't reported:** doubles quietly become `inf`.
3. **Slow for big exponents:** a loop that runs `b` times can run 2 billion times for `b = 2,000,000,000`.
4. The question says `a^b` with `b` non-negative; the code's negative exponent branch was an extra (fine as a bonus, but it hid the main issues above).
5. **0^0** should be stated explicitly: by convention `0^0 = 1`.

Fixed version (integers, overflow-safe):

```cpp
```

```cpp
#include <iostream>
#include <climits>
using namespace std;

int main() {
    long long a;
    int b;
    cin >> a >> b;

    if (b < 0) {
        cout << "Exponent must be non-negative\n";
        return 0;
    }
    if (a == LLONG_MIN) {                    // -a would overflow
        cout << "Input too large\n";
        return 0;
    }

    long long absA = (a < 0) ? -a : a;
    long long mag = 1;                       // magnitude of the answer

    if (b == 0)            mag = 1;          // a^0 = 1, including 0^0 = 1
    else if (absA == 0)    mag = 0;          // 0^b = 0 for b > 0
    else if (absA == 1)    mag = 1;          // avoids a useless 2-billion loop
    else {
        for (int i = 0; i < b; i++) {
            if (mag > LLONG_MAX / absA) {    // mag * absA would overflow
                cout << "Overflow\n";
                return 0;
            }
            mag *= absA;
        }
    }

    bool negative = (a < 0) && (b % 2 == 1); // odd power of a negative
    cout << (negative ? -mag : mag) << '\n';
}
```

Examples

Input (a b)	Output
2 10	1024
5 0	1
0 0	1 (convention)
0 5	0
-2 3	-8
2 62	4611686018427387904
2 63	Overflow

Why the loop is safe now: for `|a| ⩾ 2`, the value doubles at least every step, so overflow happens within ~63 iterations. The huge-exponent loop never runs long.

Bonus – fast exponentiation (O(log b)), a common interview follow-up:

cpp

```cpp
long long power(long long base, long long exp) {
    long long result = 1;
    while (exp > 0) {
        if (exp & 1) result *= base;    // current bit is 1 → multiply
        base *= base;                   // square the base
        exp >>= 1;                      // next bit
    }
    return result;                      // (add overflow checks for production)
}
```

`3^13`: $13 = 1101_2 \rightarrow \text{result} = 3^1 \cdot 3^4 \cdot 3^8 = 3 \cdot 81 \cdot 6561 = 1594323$ ✓

If real numbers are needed (e.g. `2.5^3`), keep your `double` version but print with `cout << fixed << setprecision(6)` (needs `<iomanip>`) or `setprecision(15)`.

Q23. Count Even and Odd Digits ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    int evenCount = 0, oddCount = 0;

    do {
        int digit = static_cast<int>(n % 10);
        if (digit % 2 == 0) evenCount++;
        else                oddCount++;
        n /= 10;
    } while (n != 0);            // do-while so input 0 counts as one even digi

    cout << "Even digits: " << evenCount << '\n';
    cout << "Odd digits: " << oddCount << '\n';
}
```

Dry run (123456): digits 6,5,4,3,2,1 → even: 6,4,2 = 3, odd: 5,3,1 = 3.

Input	Output
123456	Even 3, Odd 3
0	Even 1, Odd 0
-2468	Even 4, Odd 0 (works for negatives: `-3 % 2 == -1`, which is `!= 0`, so it goes to the odd branch)

Your `while (n != 0)` (rather than `n > 0`) is what makes negatives work.

Q24. First and Last Digit ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    int last = static_cast<int>(n % 10);   // last digit = remainder by 10
    long long temp = n;

    while (temp >= 10)                     // strip digits until one is left
        temp /= 10;

    cout << "First digit: " << temp << '\n';
    cout << "Last digit: " << last << '\n';
}
```

Dry run (95731): last = 1. temp: 95731 → 9573 → 957 → 95 → **9**.

Input	First	Last
95731	9	1
7	7	7
0	0	0
1000	1	0

Alternative (math): `first = n / pow(10, digits - 1)`, but the loop avoids floating-point `pow`.

Q25. Swap Two Numbers

Bug 🐞: The question asks for **two** methods, with a temp variable *and* without one. Only the temp version was written. Added the rest.

cpp

```cpp
#include <iostream>
#include <utility>      // std::swap
using namespace std;

// Method 1: temporary variable (clearest, always safe)
void swapWithTemp(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

// Method 2: arithmetic, no temp
void swapArithmetic(int &a, int &b) {
    a = a + b;      // a holds the sum
    b = a - b;      // b = (a+b) - b = original a
    a = a - b;      // a = (a+b) - original a = original b
}

// Method 3: XOR, no temp
void swapXor(int &a, int &b) {
    if (&a == &b) return;   // XOR-ing a variable with itself would zero it
    a ^= b;
    b ^= a;
    a ^= b;
}

int main() {
    int a, b;
    cin >> a >> b;
    int x = a, y = b;

    swapWithTemp(x, y);
    cout << "Temp:       a = " << x << ", b = " << y << '\n';

    x = a; y = b;
    swapArithmetic(x, y);
    cout << "Arithmetic: a = " << x << ", b = " << y << '\n';

    x = a; y = b;
    swapXor(x, y);
    cout << "XOR:        a = " << x << ", b = " << y << '\n';

    x = a; y = b;
    std::swap(x, y);                    // what you'd use in real code
```

```
    cout << "std::swap:  a = " << x << ", b = " << y << '\n';
}
```

Dry run (arithmetic, a=5, b=8): a = 13 → b = 13 − 8 = 5 → a = 13 − 5 = 8 ✓

Dry run (XOR, a=5=101₂, b=3=011₂): a = 110 (6) → b = 110 ^ 011 = 101 (5) → a = 110 ^ 101 = 011 (3) ✓

Method	Pros	Cons
Temp variable	Clear, safe, works for any type	Uses one extra variable
Arithmetic	No temp	`a + b` can overflow (undefined behavior for signed int)
XOR	No temp, no overflow	Integers only; breaks if both refer to the same variable
`std::swap`	Best practice	None

Interview tip: Interviewers ask this to see if you know the *trade-offs*. The temp version is what you should write in production.

Q26. Optimized Prime Check (√n) ✅

cpp

```cpp
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i <= n / i; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    }
    return true;
}

int main() {
    int n;
    cin >> n;
    cout << (isPrime(n) ? "Prime" : "Not prime") << '\n';
}
```

Why checking only up to √n works (the "explain why" part):

If `n` is composite, then `n = a × b` with `1 < a ⩽ b < n`.

- If both `a` and `b` were greater than √n, then `a × b > √n × √n = n`, a contradiction.
- So the smaller factor `a` is always **⩽ √n**.
- Therefore, if no divisor exists up to √n, none exists at all and `n` is prime.

Example (n = 36): √36 = 6. Divisor pairs: (2,18), (3,12), (4,9), (6,6). Every pair has a member ⩽ 6, so we'd find 2 immediately.

Comparison of speed (n ≈ 1,000,000,007):

Method	Checks
Up to n − 1	~1,000,000,000
Up to √n	~31,623
Up to √n, step 6	~10,500

Why `i <= n / i` and not `i * i <= n`: `i * i` can overflow `int` for `n` near 2^{31}; the division form cannot.

Why step 6: after ruling out multiples of 2 and 3, every prime is `6k ± 1`, which is `i` and `i + 2` when `i = 5, 11, 17, ...`

Q27. Print All Primes in a Range ✅

cpp

```cpp
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    int start, end;
    cin >> start >> end;

    if (start > end) {
        cout << "Invalid range\n";
        return 0;
    }

    for (int n = start;; n++) {      // loop form that is safe even when end ==
        if (isPrime(n))
            cout << n << ' ';
        if (n == end)
            break;
    }
    cout << '\n';
}
```

Why the unusual loop? `for (n = start; n <= end; n++)` is an **infinite loop / undefined behavior** when `end == INT_MAX` because `n++` overflows. Checking `n == end` before incrementing avoids that.

Examples

Input	Output
10 30	11 13 17 19 23 29
1 10	2 3 5 7
24 28	*(empty line, no primes)*
30 10	Invalid range

Complexity: O((end − start) · √end). Fine for ranges up to ~10^6.

Follow-up – Sieve of Eratosthenes (for big ranges, O(N log log N)):

cpp

```cpp
vector<bool> isP(end + 1, true);
isP[0] = isP[1] = false;
for (long long i = 2; i * i <= end; i++)
    if (isP[i])
        for (long long j = i * i; j <= end; j += i)
            isP[j] = false;
```

Q28. Prime Factorization

Bug 🐞 (cosmetic): When `n` is fully divided by small factors (e.g. `8 → 2 2 2`), the code printed a **trailing space** after the last factor. Many judges reject this. Fixed with a `first` flag so spaces go only *between* factors.

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 2) {
        cout << "Enter a number greater than 1\n";
        return 0;
    }

    bool first = true;
    auto printFactor = [&](long long f) {
        if (!first) cout << ' ';
        cout << f;
        first = false;
    };

    for (long long i = 2; i <= n / i; i++) {
        while (n % i == 0) {          // keep dividing → repeated factors
            printFactor(i);
            n /= i;
        }
    }

    if (n > 1)                        // leftover is a prime > sqrt(original)
        printFactor(n);

    cout << '\n';
}
```

Dry run (n = 84):

i	n % i == 0?	printed	n after
2	yes, yes	2, 2	21
3	yes	3	7
4	$4 \leqslant 7/4 = 1$? No → loop ends		7
after loop	$n = 7 > 1$	7	

Output: `2 2 3 7`

Input	Output
12	2 2 3
8	2 2 2
97	97
2	2

Why composites never get printed: by the time `i` reaches a composite like 4, all its prime factors (2) were already divided out of `n`.

Why the leftover `n > 1` is prime: after dividing out everything ⩽ √n, at most one prime factor larger than √n can remain.

Q29. Decimal to Binary ✅

cpp

```cpp
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    long long original = n;
    string binary;

    if (n == 0) {
        binary = "0";
    } else {
        while (n > 0) {
            binary += char('0' + n % 2);   // remainder is the next bit (LSB fi
            n /= 2;
        }
        reverse(binary.begin(), binary.end()); // bits came out backwards
    }

    cout << "Binary of " << original << " is " << binary << '\n';
}
```

Dry run (13):

n	n % 2	bits so far
13	1	1
6	0	10
3	1	101
1	1	1011

Reverse → **1101** (8 + 4 + 0 + 1 = 13 ✓).

Input	Output
10	1010
0	0
1	1
255	11111111

Special case: `n == 0` is handled separately, because the `while` loop wouldn't run and the string would be empty.

Q30. Binary to Decimal ✅

cpp

```cpp
#include <iostream>
#include <string>
#include <climits>
using namespace std;

int main() {
    string binary;
    cin >> binary;

    if (binary.empty()) {
        cout << "Invalid binary number\n";
        return 0;
    }

    long long decimal = 0;

    for (char bit : binary) {
        if (bit != '0' && bit != '1') {          // validation
            cout << "Invalid binary number\n";
            return 0;
        }

        int digit = bit - '0';

        if (decimal > (LLONG_MAX - digit) / 2) { // would decimal*2 + digit ove
            cout << "Number is too large\n";
            return 0;
        }

        decimal = decimal * 2 + digit;           // shift left, add new bit
    }

    cout << decimal << '\n';
}
```

Dry run (1101): $0 \to 0{\cdot}2{+}1 = 1 \to 1{\cdot}2{+}1 = 3 \to 3{\cdot}2{+}0 = 6 \to 6{\cdot}2{+}1 = \mathbf{13}$.

Input	Output
1101	13
0	0
00101	5 (leading zeros fine)
1021	Invalid binary number

Overflow check explained: `decimal * 2 + digit ≤ LLONG_MAX` ⟺ `decimal ≤ (LLONG_MAX - digit) / 2`. You check the inverse *before* doing the arithmetic. Correct and well done.

Reading as a string is the right choice: a binary number like `10101010101010101010` is too long to store as an `int`.

Q31. Power of Two ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    bool result = n > 0 && (n & (n - 1)) == 0;

    cout << (result ? "Power of two" : "Not a power of two") << '\n';
}
```

The trick: A power of two has exactly **one** `1` bit. Subtracting 1 flips that bit to 0 and every lower bit to 1, so the two numbers share no bits.

```
n     = 8  = 1000
n - 1 = 7  = 0111
n & (n-1)  = 0000  → power of two

n     = 6  = 0110
n - 1 = 5  = 0101
n & (n-1)  = 0100  ≠ 0 → not a power of two
```

Input	Output
1	Power of two (2^0)
16	Power of two
18	Not a power of two
0	Not a power of two (hence the `n > 0` check, because `0 & -1 == 0` would wrongly pass)
-8	Not a power of two

Complexity: O(1).

Loop alternative: `while (n % 2 == 0) n /= 2; return n == 1;` (O(log n)).

Q32. Digital Root ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    while (n >= 10) {            // repeat until a single digit remains
        long long sum = 0;
        while (n > 0) {
            sum += n % 10;
            n /= 10;
        }
        n = sum;
    }

    cout << n << '\n';
}
```

Dry run (9875): 9+8+7+5 = 29 → 2+9 = 11 → 1+1 = **2**.

Input	Output
9875	2
7	7
0	0
99	9

Math shortcut (O(1)), great to mention in interviews: For `n > 0`, the digital root is `1 + (n - 1) % 9`. This works because a number is congruent to its digit sum modulo 9.

```cpp
int root = (n == 0) ? 0 : 1 + (n - 1) % 9;
```

Check: `1 + (9875 - 1) % 9 = 1 + 9874 % 9 = 1 + 1 = 2` ✓

Q33. Right-Angled Star Triangle

Bug 🐞: The expected output is `*`, `**`, `***`, `****` (no spaces). Your code printed `"* "`, which gives `* * *` plus a trailing space. If the output is compared exactly, it fails.

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n <= 0) return 0;

    for (int i = 1; i <= n; i++) {          // row i
        for (int j = 1; j <= i; j++)        // i stars in row i
            cout << '*';
        cout << '\n';
    }
}
```

Output (n = 4):

```
*
**
***
****
```

Pattern rule: row `i` has `i` stars. Outer loop = rows, inner loop = columns in that row. This is the foundation of every pattern problem.

(If you prefer a spaced look like `* * *`, that's fine as a style choice. Just check what the problem expects.)

Q34. Inverted Star Triangle

Bug 🐞: Same issue as Q33, with an extra space after each star.

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n <= 0) return 0;

    for (int i = n; i >= 1; i--) {          // start with n stars, go down to 1
        for (int j = 1; j <= i; j++)
            cout << '*';
        cout << '\n';
    }
}
```

Output (n = 4):

```
****
***
**
*
```

Pattern rule: the outer loop counts **down**, so row `i` still prints `i` stars, but the biggest row comes first.

Q35. Number Triangle

Bug 🐞: The expected output is `1`, `12`, `123`, `1234`. Your code printed `1 2 3` with spaces and a trailing space.

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << j;                      // print the column number
        cout << '\n';
    }
}
```

Output (n = 4):

```
1
12
123
1234
```

Pattern rule: row `i` prints the numbers `1..i`, so the printed value is the **column** index `j`.

Note for interviews: with `n ≥ 10` the digits run together (`12345678910`). If asked, add a space separator or tell the interviewer you'd confirm the expected format.

Q36. Repeated-Number Triangle

Bug 🐞: Expected `1`, `22`, `333`, `4444`; your code printed `2 2` with spaces.

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << i;                      // print the ROW number
        cout << '\n';
    }
}
```

Output (n = 4):

```
1
22
333
4444
```

Q35 vs Q36 is a classic interview pair: Q35 prints the **column** `j`; Q36 prints the **row** `i`. Same loops, one variable different.

Q37. Pyramid Pattern ✅

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n <= 0) return 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < n - i; j++)     // leading spaces
            cout << ' ';

        for (int j = 0; j < 2 * i - 1; j++) // stars: 1, 3, 5, 7, ...
            cout << '*';

        cout << '\n';
    }
}
```

Output (n = 4):

```
   *
  ***
 *****
*******
```

Row formulas (the thing to memorize):

Row `i`	Spaces `n - i`	Stars `2i - 1`
1	3	1
2	2	3
3	1	5
4	0	7

You never print trailing spaces (spaces are only on the left), so this matches the expected output exactly.

Summary of Fixes (Part 2)

Q	Problem	Fix
22	`double` printed `1.04858e+06`; overflow unreported; slow loop for huge `b`; `0^0` unstated	Integer version with overflow check, `0^0 = 1`, shortcut for bases 0/1, bonus fast power
25	Only the temp-variable swap	Added arithmetic, XOR, `std::swap` and a comparison
28	Trailing space after last factor	`first` flag prints separators only between factors
33–36	Printed `"* "` / `"j "` with spaces; the expected output has none	Removed the extra space

Patterns to Remember (Part 2)

1. **Divisors come in pairs `(i, n/i)`:** loop to √n (Q20, 26, 27, 28).
2. **Use `i <= n / i`** instead of `i * i <= n` to avoid overflow.
3. **Divide out each factor fully** with a `while` loop, and the leftover > 1 is a prime (Q28).
4. `n & (n - 1)` clears the lowest set bit (Q31).
5. **Digits ↔ base conversion:** `% base` gives the next digit, `/ base` drops it (Q29); `result * base + digit` builds a number (Q30).
6. **Pattern problems:** outer loop = rows, inner loop = columns. Work out the formula per row (stars `2i-1`, spaces `n-i`) (Q33–37).
7. **Match the expected output exactly:** extra or trailing spaces can fail a test.

PART 3 – Q38 to Q40 (Final Patterns)

Legend: ✅ = correct · 🐞 = bug / spec mismatch found and fixed

#	Topic	Status
38	Diamond pattern	🐞 input meaning ("height") + format
39	Floyd's triangle	🐞 trailing space
40	Hollow square	🐞 extra spaces vs expected output

Q38. Diamond Pattern

Bugs 🐞

1. **Input meaning:** The question says "for a given **height**", but your `n` is the number of rows in the *top half* (including the middle). So entering `7` (the height in the example) prints a diamond 13 rows tall. The comment's example is 7 rows, which is `n = 4` in your code.
2. **Odd vs even height:** The question asks you to *define* and *handle consistently*. A symmetric diamond with a single-star tip and a single widest row only exists for **odd** heights. Your code never mentioned this.
3. **Format:** the example in the comment shows stars separated by spaces (`* * *`). Your code printed `***`. (Both are valid pyramids; just match whatever the problem shows. A spaced variant is below.)

Decision (state this out loud in an interview): *Height must be odd. If an even height is given, round up to the next odd number.*

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int h;
    cin >> h;

    if (h <= 0) return 0;

    if (h % 2 == 0) {                  // even → round up to odd
        h++;
        cerr << "Even height: using " << h << " instead\n";  // cerr keeps stdo
    }

    int mid = (h + 1) / 2;             // row number of the widest line

    for (int i = 1; i <= h; i++) {
        // k = how "far" row i is from the tip; grows to mid, then shrinks
        int k = (i <= mid) ? i : h - i + 1;

        for (int s = 0; s < mid - k; s++)       // leading spaces
            cout << ' ';

        for (int j = 0; j < 2 * k - 1; j++)     // stars: 1,3,5,...,2*mid-1,...
            cout << '*';

        cout << '\n';
    }
}
```

Output (height = 7):

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

Row table (h = 7, mid = 4):

Row i	k	Spaces `mid-k`	Stars `2k-1`
1	1	3	1
2	2	2	3
3	3	1	5
4	4	0	7
5	3	1	5
6	2	2	3
7	1	3	1

Input	Behavior
1	single `*`
5	5-row diamond, widest line 5 stars
6	treated as 7
0 or negative	prints nothing

The key idea: one loop over all `h` rows, with `k = min(i, h - i + 1)` style mirroring. This avoids writing two nearly identical loops (like the original did).

Your original two-loop version is also fine if you read the input as "half-height": top loop `i = 1..n`, bottom loop `i = n-1..1`. Just document that `n` is the half-height and the total height is `2n - 1`.

Variant: spaced stars (like the example in the problem comment):

cpp

```
for (int i = 1; i <= h; i++) {
    int k = (i <= mid) ? i : h - i + 1;

    for (int s = 0; s < 2 * (mid - k); s++)   // 2 spaces per missing level
        cout << ' ';

    for (int j = 0; j < 2 * k - 1; j++) {
        if (j > 0) cout << ' ';               // single space BETWEEN stars onl
        cout << '*';
    }
    cout << '\n';
}
```

Output for `h = 7`:

```
      *
    * * *
  * * * * *
* * * * * * *
  * * * * *
    * * *
      *
```

(Using `if (j > 0)` means there's no trailing space.)

Q39. Floyd's Triangle

Bug 🐞 (cosmetic): Printing `number++ << ' '` leaves a **trailing space** at the end of every row (`7 8 9 10 `). The expected output is `7 8 9 10`. Fixed by printing a space only *before* every number except the first in a row.

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    long long number = 1;            // keeps counting across rows

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            if (j > 1) cout << ' ';  // separator only between numbers
            cout << number++;
        }
        cout << '\n';
    }
}
```

Output (n = 4):

```
1
2 3
4 5 6
7 8 9 10
```

How it works: Row `i` has `i` numbers, and one counter (`number`) runs continuously through the whole triangle without resetting. This is the difference from Q35 (where the inner counter restarts each row).

Useful facts

- Total numbers printed = `n(n+1)/2`. For `n = 4` that's 10 (the last number).
- The **last number in row i** is `i(i+1)/2`; the **first number in row i** is `i(i-1)/2 + 1`.
- Using `long long` avoids overflow for big `n`.

Pattern	What's printed in row i
Q35	`1 2 … i` (restarts)
Q36	`i i … i` (row number)
Q39	next `i` numbers from a global counter

Edge case: `n <= 0` prints nothing, because the loop never runs.

Q40. Hollow Square Pattern

Bug 🐞: The expected output is:

```
****
*  *
*  *
****
```

Your code printed `"* "` (star + space) and `"  "` (two spaces) per cell, which produces `* *` `* *` with trailing spaces and the wrong width. Each cell must be **one character**: `'*'` or `' '`.

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n <= 0) return 0;

    for (int i = 1; i <= n; i++) {           // row
        for (int j = 1; j <= n; j++) {       // column
            if (i == 1 || i == n || j == 1 || j == n)
                cout << '*';                 // border cell
            else
                cout << ' ';                 // inside
        }
        cout << '\n';
    }
}
```

Output (n = 4):

```
****
*  *
*  *
****
```

Rule: a cell is a star if it's on the **first row, last row, first column, or last column**. Otherwise it's a space.

Examples

n	Output
1	`*`
2	`**` / `**`
3	`***` / `* *` / `***`
5	5×5 frame with a 3×3 empty center

Bonus (the same trick gives other shapes):

- Add `|| i == j` → hollow square with a diagonal.
- Add `|| i == j || i + j == n + 1` → hollow square with an **X**.
- Filled square: print `'*'` always.

Summary of Fixes (Part 3)

Q	Problem	Fix
38	Input `n` was half-height, not height; odd/even not defined; stars not spaced like the example	Take `height`, require odd (round even up), single loop with `k = min(i, h-i+1)`, spaced variant provided
39	Trailing space after each row	Print separator only between numbers
40	`"* "` / `"  "` cells instead of single characters	Print `'*'` or `' '`

FINAL CHEAT SHEET – All 40 Questions

Fixes made across the whole file

Q	Issue
1	Wrong text (`Hello world!` vs `Hello, World!`)
2	`int` overflow
6	Wrong input order; integers only
9	Formula instead of loop; overflow
14	Trailing space
16	Sum overflow on huge input
19	Missing `<climits>`
22	`double` output format, overflow, slow loop
25	Missing "without temp" swap
28	Trailing space
33–36	Extra spaces in patterns
38	Height vs half-height; odd/even rule
39	Trailing space
40	Extra spaces in cells

Techniques to remember

Technique	Where
`% 10`, `/ 10` (extract / drop digit)	10, 11, 15, 16, 17, 21, 23, 24, 32
`do-while` so `0` still runs once	11, 15, 16, 23
Check overflow **before** the operation	13, 14, 19, 22, 30
Loop to √n using `i <= n / i`	12, 20, 26, 27, 28
Euclid: `gcd(a,b) = gcd(b, a%b)`	18, 19
Base conversion: `% base` / `* base + digit`	29, 30
Bit trick `n & (n-1)`	31
Row/column formulas for patterns	33–40
Print separators *between* items, never after	28, 35, 39, 40

Interview habits

1. Restate the problem and **ask about edge cases** (0, negatives, huge values, ties).
2. Say the **brute force first**, then the optimization.
3. State **time and space complexity**.
4. **Dry-run** a small example out loud.
5. Match the **expected output format** exactly.